

Special Need Vehicle Rental

Developer Environment Guide — Local, Test & Production

Version: 1.0.0 **Date:** June 2026 **Applies to:** Application version 1.0.0 **Jurisdiction:** New South Wales, Australia

Table of Contents

- 1. [Environment Overview](#)
- 2. [Complete Tool & Version Reference](#)
- 3. [Local Development Environment](#)
- 4. [Local Production Simulation](#)
- 5. [Cloud Production Environment](#)
- 6. [Database Reference](#)
- 7. [Email \(SMTP\) Setup](#)
- 8. [Domain & DNS](#)
- 9. [Environment Comparison Summary](#)
- 10. [Troubleshooting](#)

1. Environment Overview

This project has three distinct environments. Each serves a different purpose and has different configuration requirements:

Environment	Purpose	Frontend	Backend	Database
Local Development	Day-to-day coding, hot reload	Vite dev server (port 5173)	Node <code>--watch</code> (port 8080)	Local SQLite file
Local Production Sim	Test the full built app before deploying	Built static files served by Node	Node (port 8080)	Local SQLite file
Cloud Production	Live public site	Built static files served by Nginx	Node via PM2 (port 8080)	SQLite on VPS disk

Why three environments?

- Local Development is fast (hot module replacement, auto-restart) but the two-server setup doesn't reflect how production works.
- Local Production Simulation runs the app exactly as it runs in the cloud — same single server, same built assets — so you can catch any build-time issues before touching the live server.
- Cloud Production is the live site, running on a DigitalOcean VPS in the Sydney region.

2. Complete Tool & Version Reference

2.1 Runtime & Build Tools

These are the tools you need installed on your development machine and on the production server.

Tool	Version	Source / Download	Notes
Node.js	20.x LTS		
	(minimum 20.11.0)	https://nodejs.org/en/download/	Use the LTS installer, not "Current". As of June 2026, Node 20 LTS is the iron release.
npm	10.x (ships with Node 20)	Bundled with Node.js	Do not install npm separately; it comes with Node.
Git	2.40+	https://git-scm.com/downloads	Required for version control and the CI/CD deploy script.
SQLite CLI	3.x (optional)	https://www.sqlite.org/download.html — or apt install sqlite3 on Linux	Only needed for manual DB inspection. The application uses better-sqlite3 internally and does not require the CLI to be present.

Important: Node.js 20 LTS is specifically chosen because **better-sqlite3** ^9.4.3 requires Node ≥ 18.x, and Node 20 is the current Long-Term Support release with security patches through April 2026. Do not use Node 18 (approaching EOL), Node 21/22 Current (not LTS — API instability risk), or older versions.

2.2 Backend npm Packages (exact versions from **package.json**)

Package	Version Spec	Resolved Purpose
express	^4.18.3	HTTP server framework
better-sqlite3	^9.4.3	Synchronous SQLite driver — chosen for simplicity; no async callback complexity
node-cache	^5.1.2	In-memory availability cache, 30-second TTL
zod	^3.22.4	Runtime schema validation on all API inputs
express-session	^1.18.0	Server-side session for admin authentication
bcrypt	^5.1.1	Password hashing for admin credentials
express-rate-limit	^7.2.0	Rate limiting: 100 req/15min on booking, 20/min on payment
nodemailer	^6.9.12	SMTP email for booking confirmations and staff alerts
pino	^8.19.0	Structured JSON logging (no PII in log bodies)
pino-http	^10.1.0	Per-request HTTP logging middleware for pino
helmet	^7.1.0	Sets common security response headers (X-Content-Type-Options, etc.)
cors	^2.8.5	Cross-Origin Resource Sharing — allows Vite dev server to call the API
Dev		
jest	^29.7.0	Unit test runner

2.3 Frontend npm Packages (exact versions from **frontend/package.json**)

Package	Version Spec	Resolved Purpose
react	^18.3.1	UI framework

Package	Version Spec	Resolved Purpose
react-dom	^18.3.1	DOM renderer for React
react-router-dom	^6.23.1	Client-side routing (SPA)
date-fns	^3.6.0	Date manipulation utilities
date-fns-tz	^3.1.3	Timezone conversion (Australia/Sydney display)
react-hook-form	^7.51.3	Installed as dependency but not actively used in forms — validation is done with native React state. Present for potential future use.
Dev		
vite	^5.2.11	Build tool and dev server
@vitejs/plugin-react	^4.3.0	Vite plugin for React JSX and Fast Refresh
tailwindcss	^3.4.3	Utility-first CSS framework
postcss	^8.4.38	CSS processing pipeline (required by Tailwind)
autoprefixer	^10.4.19	Adds vendor prefixes to CSS output
@types/react	^18.3.3	TypeScript type definitions (improves IDE autocomplete even in JS projects)
@types/react-dom	^18.3.0	TypeScript type definitions for React DOM

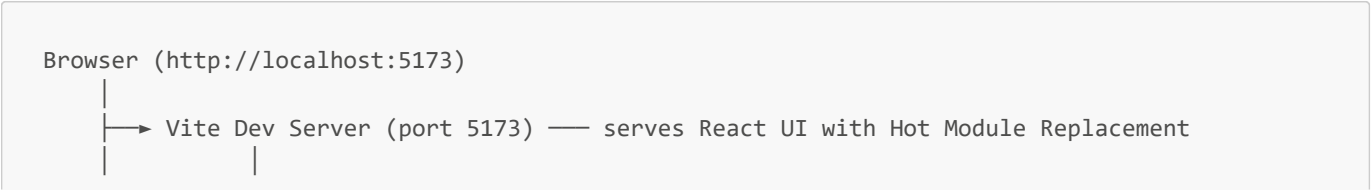
2.4 Production Server Stack

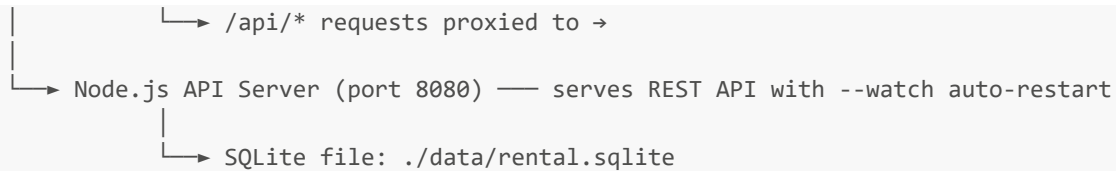
Component	Version	Source / Notes
Ubuntu	22.04 LTS (Jammy Jellyfish)	DigitalOcean droplet image. 22.04 LTS has support until April 2027. Do not use 20.04 (approaching EOL) or 24.04 (too new for stable Nginx packages at time of writing).
Nginx	1.24.x (stable)	Install from Nginx's official apt repository to get 1.24. Ubuntu's default apt repo ships 1.18.x which is older. See §5.4 for install commands.
PM2	5.x (latest 5.3.x+)	Installed globally via npm: <code>npm install -g pm2</code> . Process manager for Node.js — handles auto-restart, crash recovery, log rotation, and systemd integration.
Certbot	2.x	Installed via <code>snap</code> (recommended by Let's Encrypt) or <code>apt</code> . Manages free TLS certificates from Let's Encrypt. Auto-renews every 90 days via systemd timer.
SQLite	3.37.x (ships with Ubuntu 22.04)	No additional install needed beyond <code>apt install sqlite3</code> for the CLI. The Node driver <code>better-sqlite3</code> bundles its own SQLite library so the system SQLite version is only relevant for the CLI inspection tool.

3. Local Development Environment

3.1 What Local Development Looks Like

In development mode, two servers run simultaneously:





The Vite proxy (configured in `frontend/vite.config.js`) transparently forwards any request starting with `/api` to `http://localhost:8080`. From the browser's perspective, everything looks like it's on the same server — there are no CORS issues in normal use.

Why not just run the built frontend in dev? Because Vite's Hot Module Replacement (HMR) reflects code changes instantly without a full page reload — essential for iterating on UI. Building the frontend takes 10–30 seconds and requires a manual browser refresh, which slows development significantly.

3.2 Windows Prerequisites

These steps are for Windows 11 (the current development machine). Mac/Linux users follow the same steps substituting OS-specific installers.

Step 1 — Install Node.js 20 LTS

Download the Windows installer from: <https://nodejs.org/en/download/>

- Select: **LTS** tab → **Windows Installer (.msi)** → 64-bit
- Run the installer with default options. Make sure "Add to PATH" is checked.
- After install, open a **new** PowerShell window and verify:

```
node --version    # Expected: v20.x.x
npm --version     # Expected: 10.x.x
```

If PowerShell still says "not recognised" after install, restart the terminal — PATH changes require a new shell session.

Step 2 — Install Git

Download from: <https://git-scm.com/download/win>

- Default options are fine. Ensure "Git from the command line and also from 3rd-party software" is selected during setup.
- Verify:

```
git --version    # Expected: git version 2.4x.x
```

Step 3 — (Optional) SQLite CLI for Database Inspection

Download the precompiled Windows binary from: <https://www.sqlite.org/download.html> Under "Precompiled Binaries for Windows", download `sqlite-tools-win-x64-*.zip`. Extract `sqlite3.exe` and place it somewhere on your PATH (e.g. `C:\Windows\System32\` or a custom tools folder).

```
sqlite3 --version    # Expected: 3.4x.x
```

3.3 First-Time Setup

Open PowerShell and navigate to the project folder:

```
cd C:\Users\DueDiligence\Desktop\car_rental
```

Install backend dependencies:

```
npm install
```

This reads `package.json` and installs all 12 production packages and `jest` into `node_modules/`. This takes 1–3 minutes on first run. The result is a `node_modules/` folder and a `package-lock.json` file (do not edit this manually — it locks exact sub-dependency versions for reproducible installs).

Install frontend dependencies:

```
cd frontend
npm install
cd ..
```

This installs all Vite/React/Tailwind tooling into `frontend/node_modules/`. First run takes 2–4 minutes.

Create the database and run migrations:

```
npm run migrate
```

This runs `scripts/migrate.js` which:

1. Loads `.env` to get `DATABASE_PATH`
2. Creates the `data/` directory if it doesn't exist
3. Opens (or creates) `data/rental.sqlite`
4. Executes `migrations/001_initial_schema.sql` — creates the `vehicles`, `reservations`, and `audit_log` tables plus indexes

Expected output: `Migration complete.`

Seed 100 sample vehicles:

```
npm run seed
```

This runs `scripts/seed.js` which inserts 100 vehicles using `INSERT OR IGNORE` — meaning it is safe to run multiple times without creating duplicates. The fleet breakdown:

Group	Count	Vehicle	Hourly	Daily
Wheelchair vans	20	Toyota HiAce Wheelchair Van 1–20	AUD \$30.00	AUD \$165.00
Accessible vans	20	Toyota Tarago Accessible Van 1–20	AUD \$30.00	AUD \$165.00
People movers	20	Kia Carnival People Mover 1–20	AUD \$25.00	AUD \$132.00
Station wagons	20	Subaru Outback Wagon 1–20	AUD \$22.00	AUD \$132.00

Group	Count	Vehicle	Hourly	Daily
Sedans	20	Toyota Camry Sedan 1–20	AUD \$22.00	AUD \$132.00

Expected output: **Seeded 100 vehicles.**

3.4 Configure the `.env` File

The `.env` file at the project root controls all runtime configuration. A template is provided as `.env.example`. The development `.env` is already created with safe defaults.

For local development, the most important settings:

```
PORT=8080
NODE_ENV=development
DATABASE_PATH=./data/rental.sqlite
ADMIN_USERNAME=admin
ADMIN_PASSWORD_HASH=      ← leave blank; dev mode falls back to password "admin"
SESSION_SECRET=dev_secret_please_change_for_production_use_a_random_64_char_string
CORS_ORIGIN=http://localhost:5173,http://localhost:8080
```

Email in development: Leave all `SMTP_*` variables blank. The server silently skips email sending if `SMTP_HOST` is not set — bookings still work, they just don't send confirmation emails. This avoids needing an SMTP server during development.

Changing the admin password in development: In `NODE_ENV=development`, if `ADMIN_PASSWORD_HASH` is empty, the admin password defaults to the literal string `admin`. Do not expose this to any public network.

3.5 Running the Development Servers

You need **two terminal windows** open simultaneously.

Terminal 1 — Start the API backend:

```
cd C:\Users\DueDiligence\Desktop\car_rental
npm run dev
```

This runs `node --watch src/server.js`. The `--watch` flag is a built-in Node.js 20 feature (no nodemon needed) that automatically restarts the server when any `.js` file in the project changes.

Expected output:

```
{"level":30,"time":...,"msg":"Server running on port 8080"}
```

Terminal 2 — Start the frontend:

```
cd C:\Users\DueDiligence\Desktop\car_rental\frontend
npm run dev
```

This starts the Vite development server.

Expected output:

```
VITE v5.x.x  ready in xxx ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
```

Now open your browser to: <http://localhost:5173>

URL	What you'll see
http://localhost:5173	Landing page of the booking site
http://localhost:5173/admin/login	Admin login page
http://localhost:5173/admin	Admin dashboard (after login)
http://localhost:8080/api/v1/health	API health check JSON response
http://localhost:8080/api/v1/vehicles	Raw JSON list of all vehicles

Admin login credentials (development only):

- Username: `admin`
- Password: `admin`

3.6 Working with the Local Database

The SQLite database file lives at `./data/rental.sqlite`. This file is a single binary file — you can copy it, back it up, or delete it and re-run `npm run migrate && npm run seed` to start fresh.

To inspect the database manually:

```
sqlite3 data\rental.sqlite
```

Useful SQLite commands inside the shell:

```
.tables          -- list all tables
.schema vehicles -- show vehicles table definition
SELECT id, name, type FROM vehicles LIMIT 5;
SELECT COUNT(*) FROM vehicles;
SELECT * FROM reservations ORDER BY created_at DESC LIMIT 5;
.quit
```

To reset the database completely:

```
Remove-Item data\rental.sqlite
npm run migrate
npm run seed
```

This is safe to do at any time in development because the database is local and has no production data.

4. Local Production Simulation

4.1 Purpose

Before deploying to the live cloud server, always test the fully-built application locally. This catches:

- Build errors (TypeScript/JSX compilation failures, missing imports)
- Asset path problems (fonts, images not loading from correct paths)
- Environment variable issues that only appear under `NODE_ENV=production`
- The fact that the single Node.js server correctly serves both the API and the React static files

4.2 Build and Run

Step 1 — Build the frontend:

```
cd C:\Users\DueDiligence\Desktop\car_rental\frontend
npm run build
cd ..
```

Vite compiles all React/JSX, runs Tailwind's purge pass (removes unused CSS classes), bundles everything into `frontend/dist/`. This takes 15–40 seconds.

Expected output:

```
vite v5.x.x building for production...
✓ xxx modules transformed.
dist/index.html           x.xx kB
dist/assets/index-xxx.js  xxx.xx kB | gzip: xx.xx kB
dist/assets/index-xxx.css  xx.xx kB | gzip: x.xx kB
✓ built in x.xxs
```

Step 2 — Set `NODE_ENV` to production and start the server:

```
$env:NODE_ENV="production"
npm start
```

The server starts on port 8080. Because `frontend/dist/` now exists, the server will detect it and serve the built React files directly from Express (see `src/server.js` lines 80–88).

Step 3 — Open the browser:

Open `http://localhost:8080` — you are now running the same setup as production, except on your local machine.

Important: With `NODE_ENV=production`, the session cookie is set to `secure: true`, which means it only works over HTTPS. Logging in to `/admin` will fail locally in production mode because `http://` is not HTTPS. To test admin locally in production mode, either temporarily set `secure: false` in the session config or use a local HTTPS proxy (like `mkcert`). For most testing purposes, the regular dev mode is sufficient.

Step 4 — Reset to development mode:

```
$env:NODE_ENV="development"
```


5. Cloud Production Environment

5.1 Hosting Provider — DigitalOcean (Recommended)

Provider: DigitalOcean — <https://www.digitalocean.com>

Why DigitalOcean over AWS/Azure:

- Simpler pricing — flat monthly rate, no surprise bills from per-request charges
- Managed backups included as a simple add-on (\$2/month)
- Sydney region (SYD1) available — essential for NSW data sovereignty and low latency
- Intuitive control panel — easier for a non-technical business owner to log into and check status
- No free-tier account required; predictable cost from day one

Droplet specification:

Setting	Value	Reason
Plan	Basic (Shared CPU)	Sufficient for 100 vehicles, low concurrent traffic
CPU/RAM	1 vCPU / 1 GB RAM	SQLite + Node.js comfortably fit in 1 GB; Node process typically uses 80–150 MB
Storage	25 GB SSD	SQLite DB will never exceed 50 MB; OS + app + logs ~5 GB total
Region	Sydney (SYD1)	NSW data sovereignty; low latency for Australian users
OS Image	Ubuntu 22.04 LTS (x64)	Jammy Jellyfish — 5-year LTS support until April 2027
Backups	Enabled (~20% of droplet cost = ~\$2/month)	Automated weekly snapshots stored by DigitalOcean
SSH Key	Required — add your public key during droplet creation	Password authentication will be disabled post-setup
Monthly cost	~AUD \$10–12	DigitalOcean charges in USD (~\$6 USD); exchange rate varies

Total estimated monthly infrastructure cost:

Item	Cost (AUD/month)
Droplet (1 vCPU / 1 GB / SYD1)	~\$10
Automated backups	~\$2
Domain (.com.au — amortised monthly)	~\$2
Let's Encrypt TLS	\$0
Email (Mailgun Flex free tier)	\$0
Uptime monitoring (UptimeRobot free)	\$0
Total	~\$14/month

5.2 Initial Server Provisioning

SSH into the droplet as **root** immediately after creation:

```
ssh root@YOUR_DROPLET_IP
```

Update the OS:

```
apt update && apt upgrade -y
```

This applies all security patches. Always do this on a fresh server before installing anything else.

Create a non-root deploy user:

Running the application as **root** is a security risk. Create a dedicated user:

```
adduser deploy
usermod -aG sudo deploy

# Copy root's SSH key to the deploy user so you can SSH in as deploy
rsync --archive --chown=deploy:deploy ~/.ssh /home/deploy/
```

Now all subsequent steps should be done as the **deploy** user:

```
su - deploy
```

Disable root SSH login (security hardening):

```
sudo nano /etc/ssh/sshd_config
# Set: PermitRootLogin no
# Set: PasswordAuthentication no    ← only if you're using SSH key auth
sudo systemctl restart sshd
```

Configure the firewall (UFW):

```
sudo ufw allow OpenSSH      # port 22 – keep SSH access
sudo ufw allow 80           # HTTP (Nginx, redirects to HTTPS)
sudo ufw allow 443         # HTTPS (Nginx with TLS)
sudo ufw enable
sudo ufw status
```

Node.js on port 8080 should **not** be open to the public — Nginx proxies to it internally. The firewall blocks direct access to 8080.

5.3 Install Node.js 20 LTS

Ubuntu 22.04's default apt repository ships Node 12.x — far too old. Use the NodeSource repository to get Node 20:

```
# Add NodeSource repository for Node 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
```

```
# Install Node.js (includes npm 10.x)
sudo apt install -y nodejs

# Verify versions
node --version    # Must show: v20.x.x
npm --version     # Must show: 10.x.x
```

5.4 Install Nginx 1.24 (Stable)

Ubuntu 22.04's default `apt` ships Nginx 1.18.x. To get the current stable branch (1.24.x), add the official Nginx apt repository:

```
# Install prerequisites
sudo apt install -y curl gnupg2 ca-certificates lsb-release ubuntu-keyring

# Import Nginx signing key
curl https://nginx.org/keys/nginx_signing.key | gpg --dearmor \
  | sudo tee /usr/share/keyrings/nginx-archive-keyring.gpg >/dev/null

# Add the stable Nginx repository
echo "deb [signed-by=/usr/share/keyrings/nginx-archive-keyring.gpg] \
http://nginx.org/packages/ubuntu $(lsb_release -cs) nginx" \
  | sudo tee /etc/apt/sources.list.d/nginx.list

# Install Nginx 1.24
sudo apt update
sudo apt install -y nginx

# Verify version
nginx -v    # Expected: nginx/1.24.x
```

Enable Nginx to start on boot:

```
sudo systemctl enable nginx
sudo systemctl start nginx
```

5.5 Install PM2 (Process Manager)

PM2 keeps the Node.js server running after crashes, reboots, and deployments. It is installed globally:

```
sudo npm install -g pm2

# Verify
pm2 --version    # Expected: 5.x.x
```

5.6 Install Certbot for Let's Encrypt TLS

Let's Encrypt issues free TLS certificates that auto-renew every 90 days. Certbot is the recommended client. Install via `snap` (the method currently recommended by Let's Encrypt for Ubuntu 22.04):

```
sudo snap install --classic certbot
sudo ln -s /snap/bin/certbot /usr/bin/certbot
```

Verify:

```
certbot --version    # Expected: certbot 2.x.x
```

5.7 Install Supporting Tools

```
sudo apt install -y git sqlite3
```

- `git` — for pulling code from your repository
- `sqlite3` — CLI tool for manual database inspection and the backup script

5.8 Deploy the Application

Clone the repository:

```
sudo mkdir -p /var/www/rental
sudo chown deploy:deploy /var/www/rental
cd /var/www/rental
git clone https://github.com/YOUR_ORG/special-need-vehicle-rental.git .
```

If you are not using Git hosting (GitHub/GitLab), you can instead transfer files via `scp` or `rsync`. For example from your Windows machine:

```
# Run on your local Windows machine (PowerShell)
scp -r "C:\Users\DueDiligence\Desktop\car_rental\*"
deploy@YOUR_DROPLET_IP:/var/www/rental/
```

Create the production `.env` file:

```
cd /var/www/rental
cp .env.example .env
nano .env
```

Fill in ALL values. Critical production settings:

```
PORT=8080
NODE_ENV=production
BASE_URL=https://rentals.facilitydomain.com.au

DATABASE_PATH=/var/www/rental/data/rental.sqlite

ADMIN_USERNAME=admin
ADMIN_PASSWORD_HASH=      ← MUST set this before going live (see §5.9)
```

```
SMTP_HOST=smtp.mailgun.org    ← or your SMTP provider (see §7)
SMTP_PORT=587
SMTP_USER=postmaster@mg.facilitydomain.com.au
SMTP_PASS=your_mailgun_smtp_password
EMAIL_FROM=Special Need Vehicle Rental <noreply@facilitydomain.com.au>

FACILITY_NAME=Special Need Vehicle Rental
FACILITY_PHONE=(02) XXXX XXXX
FACILITY_EMAIL=rentals@facilitydomain.com.au
FACILITY_ADDRESS=123 Care Street, Suburb NSW 2000

SESSION_SECRET=              ← MUST generate a secure 64-character random string (see §5.9)
CORS_ORIGIN=https://rentals.facilitydomain.com.au,https://www.facilitydomain.com.au

TZ=UTC
DISPLAY_TZ=Australia/Sydney
CURRENCY=AUD
GST_RATE=0.10

RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX=100
```

Install backend dependencies (production only — excludes jest):

```
npm ci --omit=dev
```

`npm ci` (Clean Install) is preferred over `npm install` in CI/CD and production because it installs exact versions from `package-lock.json` rather than resolving `^` ranges. `--omit=dev` skips Jest and other dev tools.

Run database migration:

```
npm run migrate
```

Seed the fleet (first deploy only):

```
npm run seed
```

Only run seed on the first deployment. Running it again won't create duplicates (`INSERT OR IGNORE`), but it's unnecessary. Skip on all subsequent deployments.

Build the frontend:

```
cd /var/www/rental/frontend
npm ci
npm run build
cd ..
```

The built files are placed in `/var/www/rental/frontend/dist/`.

Create log directory:

```
sudo mkdir -p /var/log/rental
sudo chown deploy:deploy /var/log/rental
```

5.9 Generate Secure Secrets

Admin password hash:

Never store a plain-text password. Generate a bcrypt hash of your chosen admin password:

```
node -e "const bcrypt = require('bcrypt'); bcrypt.hash('YOUR_SECURE_PASSWORD_HERE', 12).then(h => console.log(h));"
```

Copy the output (starts with `$2b$12$...`) into `.env` as `ADMIN_PASSWORD_HASH=`.

Session secret:

```
node -e "console.log(require('crypto').randomBytes(64).toString('hex'));"
```

Copy the 128-character hex string into `.env` as `SESSION_SECRET=`.

Both of these must be set before the application handles any real traffic. With `NODE_ENV=production`, a missing session secret would use the insecure fallback `'dev_secret_change_me'` — this is exploitable.

5.10 Configure Nginx

Copy the template and edit it with your actual domain name:

```
sudo cp /var/www/rental/nginx.conf.example /etc/nginx/conf.d/rental.conf
sudo nano /etc/nginx/conf.d/rental.conf
```

Replace every occurrence of `rentals.facilitydomain.com.au` with your actual subdomain (e.g. `rentals.carecentre.com.au`).

Test the config before reloading:

```
sudo nginx -t
# Expected: nginx: configuration file /etc/nginx/nginx.conf test is successful
```

If the test passes, reload Nginx:

```
sudo systemctl reload nginx
```

5.11 Obtain TLS Certificate (Let's Encrypt)

Your domain's DNS A record must point to the server's IP address before running this. Let's Encrypt verifies domain ownership by making an HTTP request to the server.

```
sudo certbot --nginx -d rentals.facilitydomain.com.au
```

Certbot will:

1. Request a certificate from Let's Encrypt
2. Automatically modify your Nginx config to add the certificate paths
3. Set up a systemd timer for auto-renewal

Verify the renewal timer is active:

```
sudo systemctl status certbot.timer  
# Should show: active (waiting)
```

Test renewal (dry run — does not actually renew):

```
sudo certbot renew --dry-run
```

Certificates renew automatically every 90 days. You should not need to touch this again.

5.12 Start the Application with PM2

```
cd /var/www/rental  
pm2 start ecosystem.config.js --env production
```

This starts the Node.js process under PM2 supervision with the production environment. The process is named `rental-api`.

Configure PM2 to start on server reboot:

```
pm2 startup  
# Copy and run the command that PM2 prints (it starts with "sudo env PATH=...")  
pm2 save
```

Verify everything is running:

```
pm2 status  
# Should show: rental-api | online | ...  
  
curl http://localhost:8080/api/v1/health  
# Expected: {"status":"ok","db":"ok","uptime_seconds":...}
```

Then open your domain in a browser — the full application should be live.

5.13 Subsequent Deployments

For updates after the initial deployment:

```
cd /var/www/rental

# Pull latest code
git pull

# Update backend dependencies if package.json changed
npm ci --omit=dev

# Rebuild frontend if any frontend files changed
cd frontend && npm ci && npm run build && cd ..

# Run migrations if schema changed (safe to re-run – uses CREATE TABLE IF NOT EXISTS)
npm run migrate

# Reload the application gracefully (zero downtime)
pm2 reload rental-api
```

`pm2 reload` is preferred over `pm2 restart` because it performs a graceful reload — new requests go to the new process while in-flight requests finish on the old one, minimising downtime.

5.14 Set Up Automated Daily Backups

Create the backup script:

```
sudo nano /usr/local/bin/backup-rental-db.sh
```

```
#!/bin/bash
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
DB_PATH="/var/www/rental/data/rental.sqlite"
BACKUP_DIR="/var/backups/rental"

mkdir -p "$BACKUP_DIR"
sqlite3 "$DB_PATH" ".backup '$BACKUP_DIR/rental_$TIMESTAMP.sqlite'"

# Keep last 14 days only
find "$BACKUP_DIR" -name "rental_*.sqlite" -mtime +14 -delete
```

```
sudo chmod +x /usr/local/bin/backup-rental-db.sh
```

Schedule it via cron to run at 2:30 AM Sydney time (= 16:30 UTC):

```
crontab -e
```

Add:

```
30 16 * * * /usr/local/bin/backup-rental-db.sh
```


Test the script manually once:

```
/usr/local/bin/backup-rental-db.sh  
ls /var/backups/rental/    # Should show the .sqlite backup file
```

5.15 Set Up Uptime Monitoring

Use **UptimeRobot** (free tier, no credit card required):

1. Create an account at <https://uptimerobot.com>
2. Click "Add New Monitor"
3. Type: **HTTP(S)**
4. Friendly Name: **Special Need Vehicle Rental**
5. URL: **<https://rentals.facilitydomain.com.au/api/v1/health>**
6. Monitoring Interval: **5 minutes**
7. Alert contacts: add your email address
8. Save

UptimeRobot will email you if the health check fails — i.e. if the site goes down.

6. Database Reference

6.1 Database Engine and Location

Environment	Engine	File Location
Local Development	SQLite 3.x	<code>./data/rental.sqlite</code> (relative to project root)
Local Production Sim	SQLite 3.x	<code>./data/rental.sqlite</code>
Cloud Production	SQLite 3.x	<code>/var/www/rental/data/rental.sqlite</code>

Why SQLite and not PostgreSQL or MySQL?

For this application's scale (100 vehicles, expected <50 concurrent users, <500 bookings/month), SQLite is the correct choice because:

- Zero server cost — no RDS, Cloud SQL, or managed database fees
- Zero operational overhead — no service to start, patch, or back up separately
- The database file is a single file that can be copied directly for backup
- **better-sqlite3** v9.x uses WAL (Write-Ahead Logging) mode — multiple readers and one writer can operate concurrently without blocking
- The full database will be <5 MB for years; even at 10,000 reservations, it will not exceed 10 MB

The only scenario requiring an upgrade to PostgreSQL would be:

- Traffic growing beyond ~500 concurrent users (unlikely at this scale)
- Need for full-text search across reservation notes
- Multi-server / load-balanced deployment (SQLite cannot be shared across servers)

6.2 Schema Location

The authoritative schema is in **`migrations/001_initial_schema.sql`**. The migration runner (**`scripts/migrate.js`**) applies it using **`db.exec(sql)`** which is idempotent because every **`CREATE TABLE`** uses **`IF NOT EXISTS`**.

6.3 Inspecting the Production Database (Read-Only)

To inspect the live database without disrupting the application:

```
# SSH into the production server
ssh deploy@YOUR_DROPLET_IP

# Open the database in read-only mode
sqlite3 -readonly /var/www/rental/data/rental.sqlite

# Useful queries
SELECT COUNT(*) FROM reservations WHERE status = 'pending';
SELECT COUNT(*) FROM reservations WHERE DATE(created_at) = DATE('now');
SELECT vehicle_id, COUNT(*) as bookings FROM reservations GROUP BY vehicle_id ORDER BY
bookings DESC;
.quit
```

7. Email (SMTP) Setup

7.1 Why Email is Needed

The application sends three types of email:

1. **Booking confirmation** to the customer — contains the booking reference, dates, price, and cancellation link
2. **New booking alert** to the facility staff inbox — triggers the manual payment confirmation call
3. **Cancellation confirmation** to the customer — confirms the booking is cancelled

In Phase 1, the staff alert email is particularly important because it is how staff know to call a customer. If email is not configured, staff must manually monitor the admin dashboard for new **pending** reservations.

7.2 Recommended: Mailgun (Free Tier)

Provider: Mailgun — <https://www.mailgun.com> **Free tier:** Flex plan — 100 emails per day, no credit card required for the first 3 months; after that ~\$0.80 per 1,000 emails (well under \$1/month at this volume)

Setup:

1. Create account at mailgun.com
2. Add and verify your sending domain (e.g. **mg.facilitydomain.com.au**) — Mailgun provides DNS records to add at your registrar
3. Navigate to: Sending → Domain Settings → SMTP credentials
4. Note the SMTP hostname, port, username, and password

.env values for Mailgun:

```
SMTP_HOST=smtp.mailgun.org
SMTP_PORT=587
SMTP_USER=postmaster@mg.facilitydomain.com.au
SMTP_PASS=your_mailgun_api_key_smtp_password
EMAIL_FROM=Special Need Vehicle Rental <noreply@facilitydomain.com.au>
```

7.3 Alternative: AWS SES

Provider: Amazon Simple Email Service — <https://aws.amazon.com/ses/> **Cost:** \$0.10 per 1,000 emails (effectively free at this volume — ~60 emails/month = less than \$0.01) **Limitation:** Requires verifying your sending domain; initial sandbox mode limits recipients to verified emails only until you request production access.

```
SMTP_HOST=email-smtp.ap-southeast-2.amazonaws.com ← Sydney region endpoint
SMTP_PORT=587
SMTP_USER=AKIAIOSFODNN7EXAMPLE ← SES SMTP access key ID
SMTP_PASS=your_ses_smtp_secret
```

7.4 Testing Email Locally

For local development, use **Mailpit** (free, open-source local mail catcher) or **Mailtrap**:

Mailtrap (easiest): <https://mailtrap.io> — free plan provides a fake SMTP inbox you can send to. All emails are caught and displayed in a web UI — they never leave to real addresses.

```
SMTP_HOST=sandbox.smtp.mailtrap.io
SMTP_PORT=2525
SMTP_USER=your_mailtrap_username
SMTP_PASS=your_mailtrap_password
```

8. Domain & DNS

8.1 Domain Name

The application should be accessible on a subdomain of the facility's existing domain, for example:

- [rentals.facilityname.com.au](#) — recommended (clearly scoped to rentals)
- [hire.facilityname.com.au](#) — alternative
- [booking.facilityname.com.au](#) — alternative

A [.com.au](#) domain is appropriate for a commercial service. Annual cost is approximately **AUD \$15–20/year**.

8.2 Domain Registrars (Australia)

Registrar	Price (AUD/year)	Notes
VentralIP	~\$15	Recommended — Australian-owned, excellent local support, fast DNS propagation
Crazy Domains	~\$12–15	Popular Australian option
Netfleet	~\$18	Good for premium/auction domains
AWS Route 53	~\$2 (routing) + ~\$15 domain	Adds complexity if not already using AWS

8.3 DNS Configuration

After purchasing the domain, add the following DNS record at your registrar's DNS management panel:

Type	Name	Value	TTL
A	rentals	YOUR_DROPLET_IP	300 (5 min)

For example, if your facility domain is `carecentre.com.au` and the droplet IP is `203.0.113.1`:

```
Type: A
Name: rentals
Value: 203.0.113.1
TTL: 300
```

DNS propagation takes 5 minutes to 48 hours depending on the registrar and global DNS caches. You can check propagation at <https://dnschecker.org>.

9. Environment Comparison Summary

Setting	Local Development	Local Prod Sim	Cloud Production
Frontend served by	Vite dev server (5173)	Express static (8080)	Nginx (80/443)
API served by	Node <code>--watch</code> (8080)	Node <code>npm start</code> (8080)	Node via PM2 (8080)
Admin password	<code>admin</code> (plaintext fallback)	bcrypt hash in <code>.env</code>	bcrypt hash in <code>.env</code>
Session cookie secure	<code>false</code> (HTTP allowed)	<code>true</code> (requires HTTPS)	<code>true</code> (HTTPS via Nginx)
HTTPS	No	No (or via mkcert proxy)	Yes — Let's Encrypt
CORS origin	<code>localhost:5173,localhost:8080</code>	<code>localhost:8080</code>	<code>rentals.facilityname.com.au</code>
Email	Disabled or Mailtrap	Disabled or Mailtrap	Mailgun / AWS SES
Database path	<code>./data/rental.sqlite</code>	<code>./data/rental.sqlite</code>	<code>/var/www/rental/data/rental.sqlite</code>
Rate limiting	Active (100 req/15min)	Active	Active
HSTS header	Off	Off	On (max-age=31536000)
Log output	Console (stdout)	Console (stdout)	PM2 log files in <code>/var/log/rental/</code> + Nginx logs in <code>/var/log/nginx/</code>
NODE_ENV	<code>development</code>	<code>production</code>	<code>production</code>

10. Troubleshooting

`npm` or `node` not recognised after installation (Windows)

PowerShell caches the PATH at startup. Close the terminal completely and open a new one. If still not found, verify Node.js is in PATH: search "Environment Variables" in Windows settings → "Path" → check for a `nodejs` entry.

Port 8080 already in use

Another process is using port 8080. Find and stop it:

```
# Windows
netstat -ano | findstr :8080
# Note the PID in the last column
taskkill /PID <PID> /F
```

```
# Linux
lsof -i :8080
kill -9 <PID>
```

`better-sqlite3` fails to install (Windows)

`better-sqlite3` contains native C++ code that must be compiled during `npm install`. On Windows, this requires the Visual Studio C++ build tools. The easiest fix:

```
npm install --global windows-build-tools
```

Or install "Desktop development with C++" from the Visual Studio Installer. If that's too heavy, use WSL2 (Windows Subsystem for Linux) which runs a real Linux environment.

database is locked error

SQLite uses file-level locking. This error typically means two processes are writing to the database simultaneously. In development, check you haven't accidentally started two instances of the API server. In production, PM2 runs a single Node.js process — this error should not occur unless a runaway script is accessing the database directly. Check: `pm2 list` to confirm only one `rental-api` instance is running.

Admin login returns 401 in production

Verify the `ADMIN_PASSWORD_HASH` in `.env` is set correctly. The hash must start with `$2b$`. Also check `ADMIN_USERNAME` matches exactly (case-sensitive). Generate a fresh hash: `node -e "require('bcrypt').hash('YOUR_PASSWORD',12).then(h=>console.log(h));"` and update `.env`, then `pm2 reload rental-api`.

secure cookie not sent — admin stays logged out in production

With `NODE_ENV=production`, the session cookie requires HTTPS. If you're accessing the site over HTTP, the cookie won't be sent. Ensure Nginx is correctly redirecting HTTP to HTTPS (the config in `nginx.conf.example` does this). Check: `curl -I http://rentals.yourdomain.com.au` — should return `301 Moved Permanently` with a `Location: https://...` header.

Nginx 502 Bad Gateway

Nginx can reach the server but the Node.js API isn't responding. Check:

```
pm2 status          # Is rental-api online?
pm2 logs rental-api --lines 50    # Any startup errors?
curl http://localhost:8080/api/v1/health    # Does Node respond locally?
```

If PM2 shows the process is erroring and restarting, the most common cause is a missing or invalid `.env` value (e.g. `SESSION_SECRET` too short, database path wrong).

Let's Encrypt certificate renewal fails

```
sudo certbot renew --dry-run
# If it fails, check Nginx is running and your domain DNS is correct:
sudo systemctl status nginx
```

Certbot renews by making an HTTP challenge request to your domain. If Nginx is down, or the domain doesn't resolve to your server, renewal fails. Certbot emails the address in your Let's Encrypt account 30 days before expiry as a warning.

This guide covers version 1.0.0 of the application. Update when infrastructure or dependencies change.